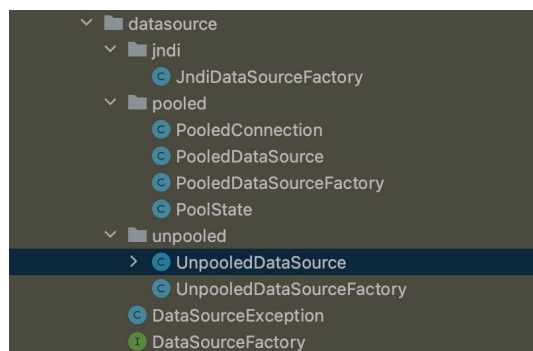




- 一、Mybatis的Unpooled
- 二、Pooled
 - 1. Connection包装成PooledConnection
 - 2. PooledDataSource负责管理PooledConnection
 - pushConnection回收连接池
 - popConnection拉取连接
 - 3.整体流程图效果
 - popConnection拉取连接
 - pushConnection回收连接池
- 三、异常情况
 - DB服务端出现db连接问题的主要情况
 - 1.默认使用UNPOOLED模式
 - 2.使用POOLED模式，仅设置maxIdle和maxActive的参数
 - 3.使用POOLED模式，设置maxIdle和maxActive的参数和poolPingEnabled与poolPingQuery
 - 客户端出现db连接问题（如k8s网络配置重置）的主要情况
 - 1.默认使用UNPOOLED模式
 - 2.使用POOLED模式，仅设置maxIdle和maxActive的参数
 - 3.使用POOLED模式，设置maxIdle和maxActive的参数和poolPingEnabled与poolPingQuery
- 四、总结

一、Mybatis的Unpooled

UNPOOLED 不使用连接池的数据源，当 dataSource 的type属性被配置成了UNPOOLED，MyBatis 首先会先实例化一个UnpooledDataSourceFactory工厂实例，然后通过getDataSource() 方法返回一个UnpooledDataSource 实例对象，具体如下，截取了部分代码。

实际使用的时候会调用getConnection() ,每调用一次就会产生一个新的 Connection 实例对象。具体方法实现如下。

[展开源码](#)

- 1.初始化驱动：判断driver驱动是否已经加载到内存中，如果还没有加载，则会动态地加载driver类，实例化后放入DriverManager注册表中，后续直接使用DriverManager即可。
 - 2.创建Connection对象：使用DriverManager.getConnection()方法创建连接
 - 3.配置Connection对象：设置是否自动提交autoCommit和隔离级别isolationLevel
- 每调用一次getConnection()方法，都会通过DriverManager.getConnection()创建新的java.sql.Connection实例

二、Pooled

1. Connection包装成PooledConnection

在pooled连接池中，mybatis将原先的Connection对象包装成PooledConnection，其中realConnection可以视为原先的Connection。

下面为PooledConnection的截取代码。

```

1
2  class PooledConnection implements InvocationHandler {
3      private static final String CLOSE = "close";
4      private static final Class<?>[] IFACES = new Class[]{Connection.class};
5      private int hashCode = 0;
6      // 所创建PooledConnection的datasource引用
7      private PooledDataSource dataSource;
8      // 真正的Connection对象
9      private Connection realConnection;
10     // 代理自己的代理Connection
11     private Connection proxyConnection;
12     private long checkoutTimestamp;
13     private long createdTimestamp;
14     private long lastUsedTimestamp;
15     private int connectionTypeCode;
16     private boolean valid;
17
18     public PooledConnection(Connection connection, PooledDataSource dataSource) {
19         this.hashCode = connection.hashCode();

```

```

20         this.realConnection = connection;
21         this.dataSource = dataSource;

24         this.valid = true;
25         // 1.动态代理创建proxyConnection
26         this.proxyConnection = (Connection)Proxy.newProxyInstance(Connection.class.getClassLoader(), IFACES, this);
27     }
28
29     public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
30         String methodName = method.getName();
31         // 当调用close()方法关闭的时候,回收此Connection到PooledDataSource中
32         if ("close".hashCode() == methodName.hashCode() && "close".equals(methodName)) {
33             this.dataSource.pushConnection(this);
34             // 执行完即结束
35             return null;
36         } else {
37             try {
38                 // 2.如果是Object中定义的方法,如toString()、hashCode()不进行check
39                 if (!Object.class.equals(method.getDeclaringClass())) {
40                     this.checkConnection();
41                 }
42
43                 return method.invoke(this.realConnection, args);
44             } catch (Throwable var6) {
45                 throw ExceptionUtil.unwrapThrowable(var6);
46             }
47         }
48     }
49     // 判断当前连接是否有效
50     private void checkConnection() throws SQLException {
51         if (!this.valid) {
52             throw new SQLException("Error accessing PooledConnection. Connection is invalid.");
53         }
54     }
55 }

```

1.动态代理创建proxyConnection：该部分使用了动态代理，实现了方法的增强，走的是invoke()，也就是任何proxyConnection的执行方法必须走该流程。实际上，在调用PooledDataSource的getConnection()方法返回的就是这个proxyConnection对象。所以在代码中session.close()实际就是使用该proxyConnection对象，具体的close()也就是pushConnection方法后面介绍。

```

@zhuming3
def withSessionAndDataMapper[T](f: DBDataMapper => T): T = {
    val (session, mapper) = genDataMapper()
    try {
        f(mapper)
    } finally {
        if (session != null) session.close()
    }
}

```

2.Object.class.equals(method.getDeclaringClass())这里要判断当前所调用的方法时候属于Object类，因为如果我们调用的方法比如hashCode()、equals等这些事继承于Object类的方法，那么mybatis就不需要进行动态代理了，直接执行父类代码即可。

2. PooledDataSource负责管理PooledConnection

PooledDataSource主要是将PooledConnection放入PoolState中进行管理

```

1 public class PooledDataSource implements DataSource {
2
3
4
5     private final UnpooledDataSource dataSource;
6     protected int poolMaximumActiveConnections = 10;
7     protected int poolMaximumIdleConnections = 5;
8     protected int poolMaximumCheckoutTime = 20000;
9     protected int poolTimeToWait = 20000;
10    protected String poolPingQuery = "NO PING QUERY SET";
11    protected boolean poolPingEnabled = false;
12    protected int poolPingConnectionsNotUsedFor = 0;
13    // 由数据库 URL、username 和 password 三部分组合计算出来的 hash 值，主要用于连接对象确认归属的连接池
14    private int expectedConnectionTypeCode;
15 }

```

poolTimeToWait: 当当前连接数达到poolMaximumActiveConnections+poolMaximumIdleConnections后，再有新的连接会等待该时间

PoolState是实际意义上连接池的体现，主要是**idleConnections**和**activeConnections**

```

1 public class PoolState {
2     protected PooledDataSource dataSource;
3     protected final List<PooledConnection> idleConnections = new ArrayList();
4     protected final List<PooledConnection> activeConnections = new ArrayList();
5     .....
6 }

```

• pushConnection回收连接池



```

def withSessionAndDataMapper[T](f:DBDataMapper=>T): T = {
    val (session,mapper) = genDataMapper()
    try {
        f(mapper)
    } finally {
        if(session!=null) session.close()
    }
}

```

session.close()后通过动态代理，实际上是调用了pushConnection方法，该方法实现了连接池回收，

```

1 protected void pushConnection(PooledConnection conn) throws SQLException {
2     synchronized(this.state) {
3         // active的List中移除该PooledConnection连接
4         this.state.activeConnections.remove(conn);
5         if (conn.isValid()) {
6             PoolState var10000;
7             // 如果idle的List大小小于设定值
8             if (this.state.idleConnections.size() < this.poolMaximumIdleConnections && conn.getConnectionTypeCod
9                 var10000 = this.state;
10                var10000.accumulatedCheckoutTime += conn.getCheckoutTime();
11                if (!conn.getRealConnection().getAutoCommit()) {
12                    conn.getRealConnection().rollback();
13                }
14                // 1.复用db连接
15                PooledConnection newConn = new PooledConnection(conn.getRealConnection(), this);
16                // 2.将newConn放入idle的List
17                this.state.idleConnections.add(newConn);
18                newConn.setCreatedTimestamp(conn.getCreatedTimestamp());
19                newConn.setLastUsedTimestamp(conn.getLastUsedTimestamp());
20                conn.invalidate();
21                if (log.isDebugEnabled()) {
22                    log.debug("Returned connection " + newConn.getRealHashCode() + " to pool.");
23                }
24                this.state.notifyAll();
25            } else {
26                var10000 = this.state;
27                var10000.accumulatedCheckoutTime += conn.getCheckoutTime();
28                if (!conn.getRealConnection().getAutoCommit()) {
29                    conn.getRealConnection().rollback();
30                }
31                // 3.调用realConnection的close方法，实际上关闭该连接

```

```

32         conn.getRealConnection().close();
33         if (log.isDebugEnabled()) {
34             log.debug("Connection " + conn.getRealHashCode() + " attempted to return to the pool, but was not in the pool.");
35         }
36         conn.invalidate();
37     }
38     } else {
39         if (log.isDebugEnabled()) {
40             log.debug("A bad connection (" + conn.getRealHashCode() + ") attempted to return to the pool, but was marked as bad.");
41         }
42         ++this.state.badConnectionCount;
43     }
44 }
45 }

```

1.复用db连接：通过conn.getRealConnection()可以获取到原连接的realConnection，再通过new PooledConnection复用该realConnection

2.将复用realConnection后的新newConn放入idle的List

3.如果空闲队列为空，或者没有设置空闲队列那么就会直接将realConnection关闭掉

• popConnection拉取连接

在具体执行sql语句的时候会调用popConnection函数，拉取db的连接

```

} else {
    // delete dirty data in trafficConsumer when batchid > commit_batchid in kafka_info db
    dataMapper.deleteForBatchId(operatorInfo)
    AggregatesSinkFunction
    .LOGGER
    .info(s"success to remove dirty date,operatorid = ${operatorInfo.operatorId}, " + s"batchid m

```

```

1
2 private PooledConnection popConnection(String username, String password) throws SQLException {
3     boolean countedWait = false;
4     PooledConnection conn = null;
5     long t = System.currentTimeMillis();
6     int localBadConnectionCount = 0;
7
8     while (conn == null) {
9         synchronized (this.state) {
10             PoolState var10000;
11             // 1.空闲队列不为空
12             if (!this.state.idleConnections.isEmpty()) {
13                 conn = (PooledConnection) this.state.idleConnections.remove(0);
14                 if (log.isDebugEnabled()) {
15                     log.debug("Checked out connection " + conn.getRealHashCode() + " from pool.");
16                 }
17             } else if (this.state.activeConnections.size() < this.poolMaximumActiveConnections) {
18                 // 2.创建新连接
19                 conn = new PooledConnection(this.dataSource.getConnection(), this);
20                 if (log.isDebugEnabled()) {
21                     log.debug("Created connection " + conn.getRealHashCode() + ".");
22                 }
23             } else {
24                 PooledConnection oldestActiveConnection = (PooledConnection) this.state.activeConnections.get(0);
25                 long longestCheckoutTime = oldestActiveConnection.getCheckoutTime();
26                 // 3.空闲队列+活跃队列都满，此时需要淘汰一个连接
27                 if (longestCheckoutTime > (long) this.poolMaximumCheckoutTime) {
28                     ++this.state.claimedOverdueConnectionCount;
29                     var10000 = this.state;
30                     var10000.accumulatedCheckoutTimeOfOverdueConnections += longestCheckoutTime;
31                     var10000 = this.state;
32                     var10000.accumulatedCheckoutTime += longestCheckoutTime;
33                     this.state.activeConnections.remove(oldestActiveConnection);
34                     if (!oldestActiveConnection.getRealConnection().getAutoCommit()) {
35                         oldestActiveConnection.getRealConnection().rollback();
36                     }
37                 }
38                 conn = new PooledConnection(oldestActiveConnection.getRealConnection(), this);
39                 oldestActiveConnection.invalidate();
40                 if (log.isDebugEnabled()) {
41                     log.debug("Claimed overdue connection " + conn.getRealHashCode() + ".");
42                 }
43             }
44         }
45         try {
46             if (!countedWait) {

```

46	++ this .state.hadToWaitCount;
47	countedWait = true ;
48	
49	
50	if (log.isDebugEnabled()) {
51	log.debug("Waiting as long as " + this .poolTimeToWait + " milliseconds for connecti
52	}
53	
54	long wt = System.currentTimeMillis();
55	this .state.wait((long) this .poolTimeToWait);
56	var10000 = this .state;
57	var10000.accumulatedWaitTime += System.currentTimeMillis() - wt;
58	} catch (InterruptedException var16) {
59	break ;
60	}
61	}
62	}
63	// 4.设置连接属性
64	if (conn != null) {
65	if (conn.isValid()) {
66	if (!conn.getRealConnection().getAutoCommit()) {
67	conn.getRealConnection().rollback();
68	}
69	
70	conn.setConnectionTypeCode(this .assembleConnectionTypeCode(this .dataSource.getUrl(), userna
71	conn.setCheckoutTimestamp(System.currentTimeMillis());
72	conn.setLastUsedTimestamp(System.currentTimeMillis());
73	this .state.activeConnections.add(conn);
74	++ this .state.requestCount;
75	var10000 = this .state;
76	var10000.accumulatedRequestTime += System.currentTimeMillis() - t;
77	} else {
78	if (log.isDebugEnabled()) {
79	log.debug("A bad connection (" + conn.getRealHashCode() + ") was returned from the pool
80	}
81	
82	++ this .state.badConnectionCount;
83	++localBadConnectionCount;
84	conn = null ;
85	if (localBadConnectionCount > this .poolMaximumIdleConnections + 3) {
86	if (log.isDebugEnabled()) {
87	log.debug("PooledDataSource: Could not get a good connection to the database.");
88	}
89	
90	throw new SQLException("PooledDataSource: Could not get a good connection to the databa
91	}
92	}
93	}
94	}
95	}
96	
97	if (conn == null) {
98	if (log.isDebugEnabled()) {
99	log.debug("PooledDataSource: Unknown severe error condition. The connection pool returned a null c
100	}
101	
102	throw new SQLException("PooledDataSource: Unknown severe error condition. The connection pool returned
103	} else {
104	return conn;
105	}
106	}

- 1.如果空闲队列不为空的话，此时会从空闲队列中取出第一个，并且返回。
- 2.如果没有设置空闲队列，或者是空闲队列目前为空。那么会判断激活队列的长度是否大于设定值，如果还有空间的话会通过**this.dataSource.getConnection()**创建全新的连接。
- 3.当空闲队列+激活队列都满，也就是大于**poolMaximumIdleConnections+poolMaximumActiveConnections**，此时连接数也满了,不能创建新的连接. 找到最老的那个,检查它是否过期。此时需要计算它的校验时间，如果校验时间大于连接池规定的最大校验时间，则认为它已经过期了。利用这个PoolConnection内部的**realConnection**重新生成一个PooledConnection
- 4.在得到连接后，需要该连接的属性进行设置。**setConnectionTypeCode**的参数意义是该标识是由数据库 URL、username 和 password 三部分组合计算出来的 hash 值，主要用于连接对象确认归属的连接池。

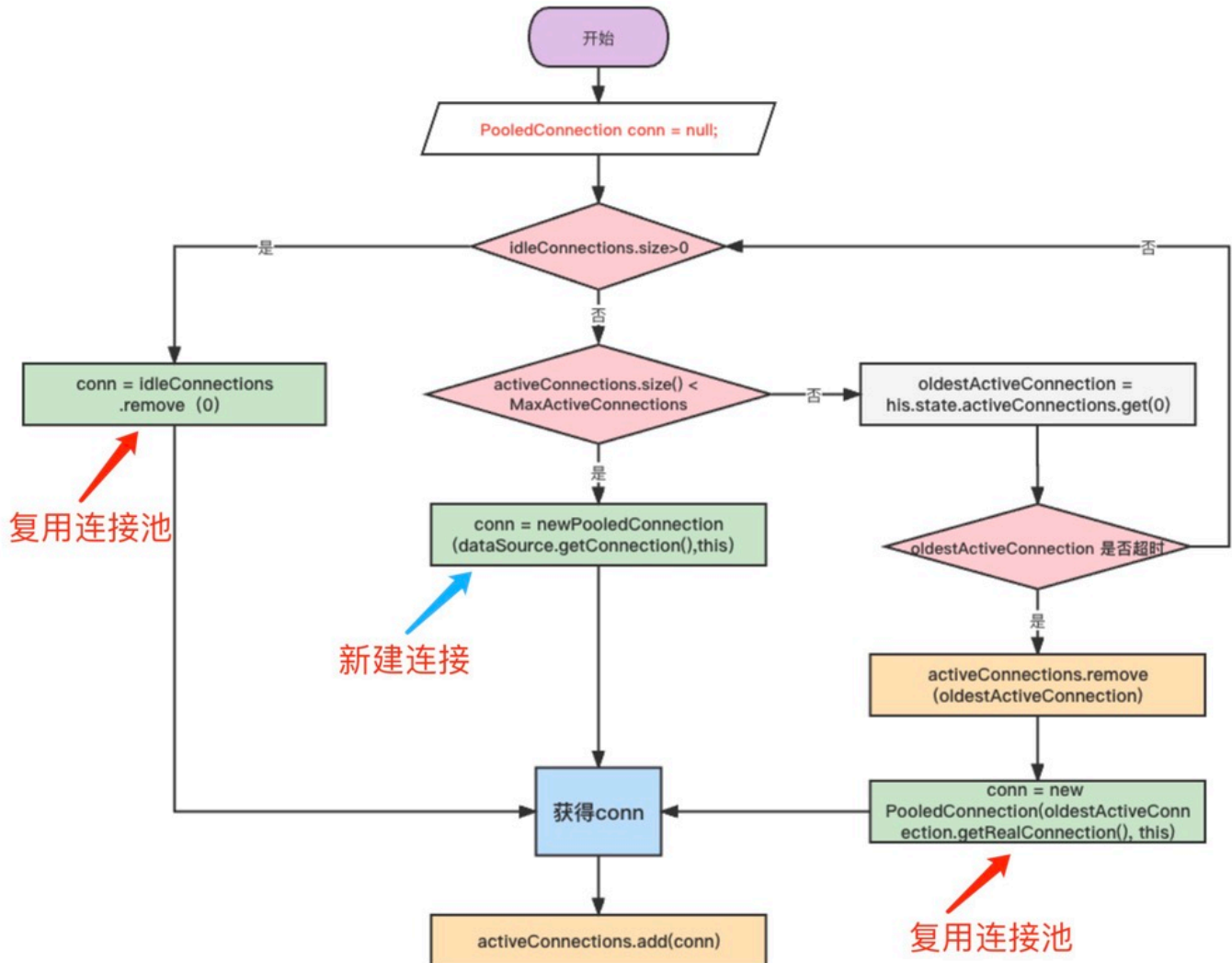
3.整体流程图效果

下面的sql为例

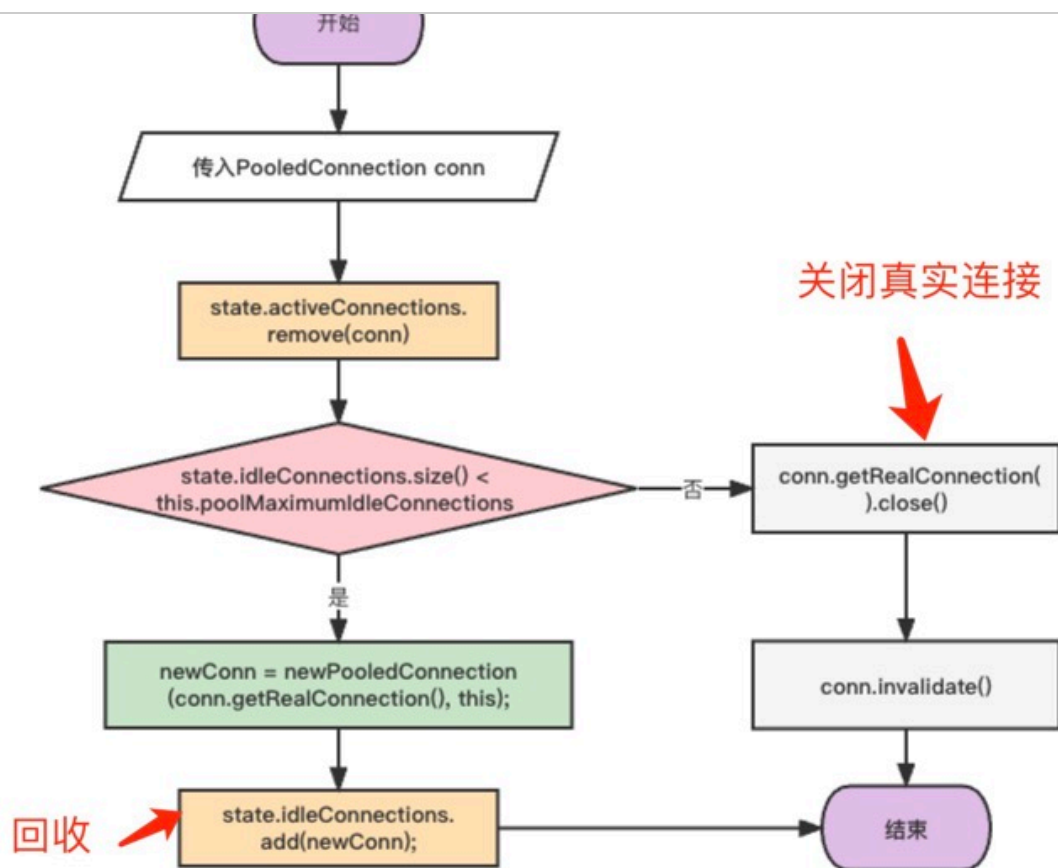
```
it("should have inserted the records to the table") {
    popConnection
}
```

```
session.commit(true)
session.close() ← pushConnection
}
```

- **popConnection** 拉取连接



- **pushConnection**回收连接池



三、异常情况

当出现客户端、服务端或者网络出现问题的时候，db的连接不稳定，此时数据可能会出现丢失，这里主要介绍下mybatis对于坏连接的处理逻辑。

在每次调用session.close()方法后，也就是源码中的pushConnection后，会调用一次pingConnection方法。（前提是需要开启poolPingEnabled开关）

```

<property name="poolPingEnabled" value="true"/>
<property name="poolPingQuery" value="select 1 from kafka_info_hour limit 1"/>

```

```

it("should have inserted the records to the table") {
    val entries = map.entrySet().toList
    mapper.batchInsert(entries.map(entry => (entry.getk
    session.commit(true)
    session.close() ← pushConnection
}

```

```

protected boolean pingConnection(PooledConnection conn) {
    boolean result = true;
    // 1. 首先判断当前连接的realConnection是否主动关闭，如果关闭那么标记当前连接断开
    try {
        result = !conn.getRealConnection().isClosed();
    } catch (SQLException var8) {
        if (log.isDebugEnabled()) {
            log.debug("Connection " + conn.getRealHashCode() + " is BAD: " + var8.getMessage());
        }
    }
    result = false;
}

if (result && this.poolPingEnabled && this.poolPingConnectionsNotUsedFor >= 0 && conn.getTimeElapsedSinceLastUse()
    try {
        if (log.isDebugEnabled()) {
            log.debug("Testing connection " + conn.getRealHashCode() + " ...");
        }
        // 2. 尝试调用当前的realConnection，并执行设定的poolPingQuery的sql语句
    }
}

```



```
Connection realConn = conn.getRealConnection();
Statement statement = realConn.createStatement();

statement.close();
if (!realConn.getAutoCommit()) {
    realConn.rollback();
}

result = true;
if (log.isDebugEnabled()) {
    log.debug("Connection " + conn.getRealHashCode() + " is GOOD!");
}
} catch (Exception var7) {
    log.warn("Execution of ping query '" + this.poolPingQuery + "' failed: " + var7.getMessage());
    // 3.执行poolPingQuery失败, 关闭realConnection, 并返回false
    try {
        conn.getRealConnection().close();
    } catch (Exception var6) {
    }

    result = false;
    if (log.isDebugEnabled()) {
        log.debug("Connection " + conn.getRealHashCode() + " is BAD: " + var7.getMessage());
    }
}
}

return result;
}
```

需要注意的是，如果当前的realConnection确实已经没有连接，那么ActiveConnections队列并不会将该连接移除，同时将badConnectionCount参数自增1，表示当前连接池的坏连接个数。

默认 `badConnectionCount > this.poolMaximumIdleConnections + 3` 的时候，线程池会抛出异常，此时线程池处于不可用状态。

DB服务端出现db连接问题的主要情况

1.默认使用UNPOOLED模式

服务端出现问题：此时每条查询都是单个连接，单条查询失败就会报错，并且失败后连接回收。
等到服务端问题解决：恢复正常。

2.使用POOLED模式，仅设置maxIdle和maxActive的参数

服务端出现问题：此时每条查询都是基本复用原先连接，如果单条查询失败会报错，但mybatis无法感知是否是mysql TCP连接问题，同时会将该连接放回池中，所以后续使用到该连接的sql依旧会报错。
等到服务端问题解决：恢复正常。

3.使用POOLED模式，设置maxIdle和maxActive的参数和poolPingEnabled与poolPingQuery

服务端出现问题：此时每条查询都是基本复用原先连接，如果单条查询失败会报错，由于设置poolPingEnabled和poolPingQuery，此时mybatis会主动查询mysql TCP连接问题，如果有问题标记错误并回收。
等到服务端问题解决：恢复正常。

客户端出现db连接问题（如k8s网络配置重置）的主要情况

1.默认使用UNPOOLED模式

客户端出现问题：此时每条查询都是单个连接，单条查询失败就会报错，并且失败后连接回收。
客户端问题解决：恢复正常。但当系统中坏连接达到一定数量就会单独报错。

2.使用POOLED模式，仅设置maxIdle和maxActive的参数

客户端出现问题：此时每条查询都是基本复用原先连接，如果单条查询失败会报错，但mybatis无法感知是否是mysql TCP连接问题，同时会将该连接放回池中，所以后续使用到该连接的sql依旧会报错。
客户端问题解决：无法恢复正常，坏连接始终存在，sql始终会报错，需手动重启。

3.使用POOLED模式，设置maxIdle和maxActive的参数和poolPingEnabled与poolPingQuery

客户端出现问题：此时每条查询都是基本复用原先连接，如果单条查询失败会报错，由于设置poolPingEnabled和poolPingQuery，此时mybatis会主动查询mysql TCP连接问题，如果有问题标记错误，并回收。

四、总结

- 1. 无论何种情况，使用mybatis连接池后，连接数永远小于等于poolMaximumIdleConnections+poolMaximumActiveConnections。
- 2. 当sql执行完毕后，mybatis连接池最多留下poolMaximumIdleConnections个连接，超出部分直接关闭连接。也就是mybatis池的概念对应着IdleConnections。
- 3. IdleConnections中连接的生命周期与 autoReconnect=true 和服务器端的wait_timeout这个配置有关，有时会有“CommunicationsException: The last packet successfully received from the server was xxx milliseconds ago”这样的报错，因为这个连接两次访问数据库的时间超出了服务器端wait_timeout的时间限制。以dup的tidb来看，默认是8个小时。所以对于dup来说IdleConnections的空闲时生命周期最大8小时，超过8小时就会报错。

```
Database changed
mysql> show global variables like '%timeout%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| connect_timeout | 10 |
| delayed_insert_timeout | 300 |
| innodb_flush_log_at_timeout | 1 |
| innodb_lock_wait_timeout | 50 |
| innodb_rollback_on_timeout | 0 |
| interactive_timeout | 28800 |
| lock_wait_timeout | 31536000 |
| net_read_timeout | 600 |
| net_write_timeout | 600 |
| rpl_semi_sync_master_timeout | 28800 |
| rpl_stop_slave_timeout | 31536000 |
| slave_net_timeout | 3600 |
| tidb_stats_load_pseudo_timeout | false |
| wait_timeout | 28800 |
+-----+-----+
14 rows in set, 1 warning (0.01 sec)
```

还有两个重要的timeout参数如下

jdbc:mysql://10.202.233.20:4000/etl_rtlp?autoReconnect=true&connectTimeout=60000&socketTimeout=60000&jdbcCompliantTruncation=false

connect_timeout是mysql建立连接的超时时间，一旦建立mysql socket连接后不会再生效。

socketTimeout由于TCP的连接方式，socket 没有办法检测到网络错误，所以socket 必须设置超时时间。一般来说主要针对慢查询，如果一个sql执行超过100s，我们设置socketTimeout=100，那么在执行到第100s时会触发报错，sql失败。

无标签